Optimization of Empty Container Repositioning by Integrating Large Language Models and Multi-Agent Systems

Kangzhen Peng[a], Shida Xu[b], Yuhong Wang[c], Mengying Hua[d], Zhihong Jin[a,*]

[a]*College of Transportation Engineering, Dalian Maritime University, No. 1 Linghai Road, Dalian, 116026, Liaoning, China*
[b]*College of Economics and Management, Dalian Jiaotong University, , Dalian, 116028, Liaoning, China*
[c]*Faculty of Maritime and Transportation, Ningbo University, No. 169, Qixing South Road, Ningbo, 315211, Ningbo, China*
[d]*Sinotrans Limited, No. 85 Renmin Road, Dalian, 116001, Liaoning, China*

Abstract

The problem of empty container repositioning is a core challenge within the container supply chain domain. This study addresses the limitations of existing solvers that depend on manual modeling and problem-specific customization by designing SOP-MAS (Standard Operating Procedure–Multi-Agent System), a multi-agent collaborative framework grounded in artificial intelligence large language models. This framework interprets the intricate business scenarios associated with the empty container repositioning problem through natural language instructions and employs the Gurobi solver to achieve end-to-end optimization. To validate the efficacy of the proposed method, benchmark tests were conducted utilizing the ComplexOR and NLP4LP datasets. The findings exhibit that our approach enhances the accuracy rate to 86.59% in comparison to traditional baseline methods. Implementing this method for an empirical analysis within the Liaoning coastal port cluster-Northeast hinterland, the system optimized repositioning routes and decreased total costs by 12.18%. This verifies its practical applicability in facilitating intelligent decision-making for the shipping industry. The research outcomes have the potential to offer

*Corresponding author. Tel.: +86; fax: +86.

Email addresses: pengkangzhen@dlmu.edu.cn (Kangzhen Peng), xushida@djtu.edu.cn (Shida Xu), wangyuhong@nbu.edu.cn (Yuhong Wang), huamengying@sinotrans.com (Mengying Hua), jinzhihong@dlmu.edu.cn (Zhihong Jin)

URL: https://orcid.org/0000-0003-4342-1561 (Zhihong Jin)

shipping companies scalable intelligent decision support.

Keywords:
large language model, multi-agent system, end-to-end optimization, mixed-integer programming, container supply chain, empty container repositioning

1. Introduction

Empty Container Repositioning (ECR), the logistical challenge of repositioning surplus empty containers from locations with excess supply to locations with unmet demand, is a central problem in container supply chain management. Optimizing ECR requires multi-period, multi-node, and multi-modal transportation decisions under dynamic supply–demand imbalances, a task commonly formulated as a Mixed-Integer Program (MIP). In practice, however, translating such complex business scenarios into a correct MIP formulation is heavily dependent on domain experts, a procedure that is not only arduous and time-consuming but also results in inflexible models that inadequately respond to dynamically evolving business requirements. Despite the fact that algebraic modeling languages (e.g., AMPL, GAMS) have marginally enhanced modeling efficiency, their reliance on expert-authored algebraic expressions means they cannot bridge the gap between unstructured business descriptions and MIP formulation without human intervention.

Recent advancements in Large Language Models (LLMs), particularly in the realms of natural language understanding and code generation, present transformative potential for the automation of the modeling process. These models' substantial in-context learning and natural language comprehension capabilities allow them to discern business intents, such as "minimize total cost" and initially generate corresponding mathematical formulations or solution code, thereby substantially reducing the complexity associated with modeling. Nevertheless, the inherent "hallucination" issue of LLMs and their lack of reliability in executing complex multi-step logical reasoning render them challenging to trust directly in operations research (OR) tasks that demand mathematical precision. As probabilistic generative models, LLMs might produce output that appear plausible but are mathematically unsound, which considerably restricts their practical application in critical decision-making systems. Empirical studies have documented characteristic failure modes of single-agent LLMs in optimization tasks: omitting decision variables or constraints that are implicit in the problem description, generating syntactically valid but logically incorrect mathematical formulations, and producing solver code that compiles yet yields infeasible or suboptimal solutions. These errors are particularly difficult to detect in the ab-

sence of an independent verification mechanism, as the same agent that generates the model is also responsible for validating it.

To address these challenges, combining the reasoning capabilities of LLMs with the collaborative strengths of Multi-Agent Systems (MAS) offers a promising approach. MAS handle complexity through task decomposition and distributed collaboration. Against this background, this paper asks the following central question: How can an LLM-driven multi-agent framework be designed to understand practical business intents while ensuring reliability, controllability, and interpretability through workflow orchestration and internal verification across the full transition from natural language to final solutions? Addressing this question can help establish a general framework for automated and adaptive large-scale optimization. We employ the ECR problem introduced above as an empirical testbed, as its multi-stakeholder, multi-modal complexity and the dual demand for mathematical rigor and business interpretability make it a compelling representative of the class of problems that the proposed framework is designed to address.

The contributions of this paper are threefold: 1) The introduction of workflow orchestration within LLM-driven multi-agent collaboration, shifting from static code generation to dynamic, adaptive task planning, thereby overcoming the rigid collaboration rules characteristic of traditional MAS; 2) The design of a systematic set of constraints and verification mechanisms to mitigate LLM hallucinations, thereby providing a novel technical path for the reliable application of LLMs in precision-critical computing domains through the integration of "multi-agent division of labor, standardized operating procedures, and exception backtracking mechanisms"; 3) The implementation and empirical validation of an end-to-end framework that translates natural language business descriptions into optimized solutions, demonstrated through the sea-land coordinated ECR problem faced by shipping companies in the Liaoning coastal port cluster, which achieved a 12.18% reduction in total operational costs and thereby verifies both the method's effectiveness and its potential for the digital and intelligent transformation of the port and shipping industry.

2. Literature Review

This research focuses on a category of problems that are essential and extensively applied within the field of OR: the automated modeling and resolution of MIP problems. This section provides a review of pertinent advancements that are central to this core theme, establishing the foundation for the subsequently proposed paradigm, which employs container SCM as a representative application scenario. We organize this review in two parts. We first survey the empty container repositioning challenges

that motivate this work. We then trace the evolution of MIP modeling paradigms used to address such problems, from the expert-driven traditional approach, through the emerging paradigm in which a *single* LLM handles the entire modeling-to-solving pipeline, to the multi-agent collaborative paradigm on which our work builds.

2.1. Overview of Empty Container Repositioning Challenges

The issue of ECR presents a significant challenge within the domain of the container supply chain, and has long been an important research topic in academia. The broader trend of supply chain digitalization [1] further accentuates the need for agile, data-driven repositioning decisions. Traditionally, ECR research has concentrated on a single mode of repositioning between seaports via maritime routes, emphasizing the optimization of components such as ship routing [2, 3, 4], shipping network design [5, 6, 7], and slot allocation [8] through MIP. Yu et al. [9] explored the dynamics of pricing competition in maritime transport under blockchain technology and ECR, determining perceived container leasing prices through inverse optimization methodologies. These investigations tend to disregard the particularities associated with land transport – specifically, when shipping companies utilize their own fleets, the marginal cost of sea repositioning approaches zero, starkly contrasting with the costly nature of land transport. Inland container transportation, which encompasses the terrestrial component of the transport chain, significantly drives costs in container shipping, accounting for 50%-70% of total container transport volume and potentially up to 40% of total transport costs, thus constituting a critical bottleneck in the pursuit of operational optimization within the shipping sector. Research on inland ECR has gradually increased in recent years. Fazi et al. [10] studied a hinterland transport network consisting of a seaport and a dry port, devising an ECR model that accounts for both laden and empty container transport to minimize costs, showing that the utilization of dry ports can reduce laden container transport costs. Cai et al. [11] investigated ECR patterns between port nodes and inland depot nodes within their respective zones, providing insights applicable to practical ECR challenges. As inland dry ports evolve, they, alongside seaports, form intermodal systems facilitated by railway collaboration; however, these entities frequently belong to disparate organizations (such as railway companies and shipping companies), which function with independent decision-making processes and competitive interactions. This decentralized nature of ECR networks, where sensitive operational data (e.g., container inventories and transport schedules) must be shared across organizational boundaries, underscores the importance of privacy-preserving and trust mechanisms for effective multi-stakeholder collaboration [12]. Luo and Chang [13] and Xie et al. [14] addressed ECR issues from the perspective of sea-rail intermodal transport, by

establishing multimodal container repositioning systems with nodes including dry port and seaport. They analyzed benefit distribution between railway companies operating at dry ports and liner shipping companies operating at seaports, under both centralized and decentralized decision-making systems, through numerical exemplifications. The extant body of research predominantly operates within a framework governed by either seaports or dry ports, managing repositioning initiatives via either seaports or dry ports, and addressing sea and land nodes in isolation. In reality, within the hinterland collection and distribution network, dry ports serve as extensions of seaports, and their collaboration can effectively diminish the detention rates of empty containers. Resolving such challenges in practice requires casting them as solvable Mixed-Integer Programs; the paradigms for modeling and solving these programs are reviewed below.

2.2. Traditional MIP Solution

The resolution of MIP problems generally adheres to a systematic OR framework that encompasses five stages. Initially, a mathematical model is constructed to abstract the real-world issue. Subsequently, an analysis of the problem's properties is undertaken to identify an appropriate solution strategy. The third stage involves the design and evaluation of algorithm performance. The fourth phase encompasses programming tasks to invoke optimization solvers and the execution of numerical simulations. Finally, the effectiveness of the solution is assessed, followed by its implementation. A core bottleneck within this framework lies in the significant dependence on experts for the laborious and time-intensive responsibilities of model development and parameter tuning.

While MIP is theoretically and algorithmically mature, its practical application encounters fundamental challenges. The modeling phase, serving as the cornerstone of the entire process, continues to present considerable difficulties. This phase necessitates experts to convert complex real-world scenarios into mathematical variables, objective functions, and constraints, which is not only resource-intensive and protracted but also significantly reliant on expert experience [15]. This dependency constitutes the fundamental impediment to its extensive and rapid adaptation to novel scenarios. Models developed for specific context often require expert re-evaluation and modification when business conditions alter (e.g., incorporating new warehouse capacity constraints), which suggests a limited capacity for generalization. As real-world issues increase exponentially in scale and complexity, the traditional method reliant on individual experts for modeling proves inadequate to satisfy these demands.

In response to the challenging nature of employing general-purpose programming languages for complex MIP models, Algebraic Modeling Languages (AMLs)

have emerged. Prominent examples, such as AMPL and GAMS, aim to elevate the level of abstraction from procedural code to algebraic expressions, thereby decoupling models from data and enhancing modeling efficiency. These languages serve as structured intermediate representations that formalize optimization models in a solver-agnostic manner, and modern systems such as SOP-MAS can leverage them as compilation targets for automatically generated code. In recent developments, optimization libraries grounded in general-purpose programming languages (e.g., Pyomo, PuLP) are embedded as libraries within modern high-level languages, facilitating the use of robust data science ecosystems and toolchains for the seamless integration of optimization models and solvers. More recently, knowledge- and data-driven hybrid methods have begun to automate scheduling and optimization beyond purely expert-built models [16].

Nonetheless, AMLs remain unable to automatically transform unstructured business descriptions into MIP formulations. For instance, within the context of container supply chain optimization, changes in the business environment necessitate experts to manually reinterpret new requirements, such as converting the need to "avoid empty container accumulation at ports" into balance constraints involving inventory variables. This demonstrates the significant reliance of AMLs on expert modeling. Additionally, the specific syntax of AMLs acts as an impediment for domain experts without technical backgrounds (e.g., shipping company schedulers), thus restricting the broad adoption of optimization technologies.

2.3. Single-Agent LLM Paradigm in Scientific Computing and OR

The emergence of LLMs in recent years brings revolutionary opportunities to the domains of scientific computing and OR. These models, trained on extensive corpora, demonstrate powerful capabilities in natural language understanding, code generation, and mathematical reasoning. LLMs offer an innovative approach to address the previously mentioned challenges. Their exceptional performance in open-domain natural language understanding significantly enhances efficacy in tasks involving arithmetic, commonsense, and symbolic reasoning. Chain-of-Thought (CoT) prompting, initially proposed by Wei et al. [17], enables models to deconstruct complex problems into multiple steps, substantially improving LLMs' reasoning ability without the need for fine-tuning, thus allowing them to tackle mathematical word problems and commonsense reasoning effectively. The introduction of CoT suggests that LLMs are also capable of resolving complex mathematical problems requiring precise, step-by-step calculation. Moreover, CoT contributes to the interpretability and reliability of LLMs. Traditionally, large-scale unsupervised deep learning models have been perceived as opaque entities with enigmatic decision-making processes,

which raises concerns about the credibility of their results. By decomposing logical reasoning problems into sequentially solvable steps, CoT provides a transparent logical framework for the results generated, thus offering a level of explicability. Research based on CoT has expanded considerably. The conceptualizations of Tree of Thoughts Yao et al. [18] and Graph of Thoughts Besta et al. [19] have further enhanced reasoning capability by allowing LLMs to explore and integrate thoughts in a structured manner. These studies demonstrate the potential of LLMs in resolving complex problems, especially those involving reasoning and mathematics.

Concurrently, the utilization of LLMs in scientific computing is transitioning from proof-of-concept to practical application, showing growing potential in disciplines such as computer science, transportation, logistics, and SCM. For example, in the domain of complex system simulation, Li et al. [20] developed a natural language interface employing adaptive dialog agents that comprehend user intent and dynamically generate task flows, thereby demonstrating the capacity of LLMs to simplify the complexity associated with simulation modeling. In the specific context of OR, the natural language processing capability of LLMs offers a novel avenue to overcome the modeling bottleneck associated with MIP. Taking the representative MIP problem of vehicle routing as an example, Jiang et al. [21] noted that LLMs are capable of precisely translating business descriptions such as "minimize total transportation cost," "ensure inventory balance across warehouses," or "meet customer time window constraints" into structured decision variables, objective functions, and constraints. This capacity provides strong support for rapid prototyping and model iteration, thereby significantly enhancing modeling efficiency. Liu et al. [22] employed LLMs for traffic flow prediction, while Ishibashi and Nishimura [23] concentrated on code generation. For modeling new scenarios, LLMs have the capability to automatically generate complete optimization models and solving code on the basis of scenario descriptions provided by algorithm engineers, in consequence significantly reducing development timelines. Parallel efforts explore the integration of Transformer architectures with reinforcement learning for general decision-making [24] and the fine-tuning of LLMs for combinatorial optimization [25], further expanding the application frontier of LLM-driven optimization.

In spite of their considerable potential, LLMs operating as solitary agents encounter the following challenges when handling complex optimization tasks: 1) Hallucination and Uncontrollability: LLMs are inherently probabilistic generative models, and as such, they may produce code that appears plausible but is mathematically invalid, leading to challenges in ensuring the accuracy and controllability of results [26]. 2) Deficiency in Transparency and Interpretability: Numerous LLMs function as "black box" models, rendering it challenging to understand their decision-making

processes and to verify results in critical decision-making scenarios. 3) Constraints in Multi-step Task Handling: The computational and decision-making demands placed on an individual LLM can increase dramatically when engaging with MIP tasks that necessitate complex, multi-step reasoning and planning [27].

2.4. Distributed Collaboration and Optimization in MAS

In response to the constraints of *single-agent* LLMs documented above, researchers have commenced investigations into technical pathways that integrate LLMs with MAS. The latter can effectively handle complex problems by leveraging multiple autonomous, interacting agents working collaboratively. Research focusing on LLM-based multi-agent collaboration has revealed that dynamic interaction, characterized by both cooperation and competition, frequently leads to unanticipated efficacy. Wu et al. [28] proposed the AutoGen framework to facilitate the implementation of LLM applications through the configuration of multiple agents. Empirical studies have demonstrated the framework's effectiveness across diverse applications, such as mathematical problem-solving and coding, conversational question answering, operational optimization, and online decision-making. Hong et al. [29] introduced Standard Operating Procedures (SOP) into MAS to facilitate software development. Within the domain of operations optimization, existing studies have engaged LLMs in addressing problems such as linear programming and MIP. For example, Zhang et al. [30] attempted to resolve the Traveling Salesperson Problem utilizing LLM based reasoning, demonstrating commendable performance on small-scale instances. Ramamonjison et al. [31] introduced the NL4Opt competition, which aimed at the extraction of meaning and formulation of optimization problems derived from natural language, involving sub-tasks of identifying semantic labeling and generating logical representations. The competition has the potential to enhance the interactive capability between non-expert users and optimization solvers. Ahmaditeshnizi et al. [32] designed a system intended for formulating and resolving (Mixed Integer) Linear Programming problems based on natural language descriptions. This system is proficient in developing mathematical models and writing debugging solver code. Xiao et al. [33] devised a multi-expert collaboration framework, known as Chain-of-Experts (CoE), aimed at automating the modeling and programming process for complex OR problems, ultimately diminishing dependency on domain-specific experts. Experimental results showed that CoE significantly outperformed state-of-the-art LLM methods in terms of performance on both LPWP and ComplexOR benchmarks.

In summary, the key to addressing the above issues lies in deeply integrating the strengths of LLMs and MAS to construct a new paradigm driven by LLMs and pow-

ered by collaborative MAS. In this paradigm, the LLM acts as a high-level planner, receiving natural language descriptions of business problems, leveraging its capabilities for abstraction and planning to automatically generate structured mathematical models, as well as MAS task allocation and collaboration rules, thereby fundamentally overcoming the cognitive dependency bottleneck inherent in traditional modeling. While recent works such as OptiMUS [32] and CoE [33] have demonstrated promising results in automating the formulation of linear and MIP from natural language, they primarily focus on syntactic correctness and benchmark performance on standardized datasets (e.g., NLP4LP, ComplexOR). In contrast, our work advances beyond proof-of-concept automation toward industrial-grade deployability in a high-stakes vertical domain, maritime logistics.

1) **Problem Context:** Whereas existing methods target generic or synthetic optimization problems, we address ECR, a real-world, multi-node, multi-modal operational challenge characterized by dynamic imbalances, contractual constraints, and significant cost implications.

2) **Workflow Architecture:** OptiMUS employs a single LLM with iterative refinement; CoE uses a fixed pipeline of expert agents. Our SOP-MAS framework introduces dynamic workflow orchestration, where agent activation is adaptively determined by problem state and verification outcomes, enabling context-aware collaboration rather than rigid sequencing.

3) **Verification and Human Integration:** Prior approaches rely heavily on solver feedback or automated unit tests. We embed a dual-layer validation mechanism: (i) programmatic constraint checking by the Testing Engineer Agent, and (ii) structured human-in-the-loop approval from domain experts, with explicit logging and backtracking support. This ensures not only mathematical feasibility but also business acceptability, a critical requirement absent in most academic benchmarks.

Thus, building upon recent advances in LLM-based optimization, our framework represents a concrete step toward industrial deployment by integrating dynamic multi-agent collaboration, auditable verification, and domain-specialized agents for maritime container logistics.

3. Methodology

This section presents the SOP-MAS framework, grounded in the Standard Operating Procedure (SOP) paradigm. The suitability of SOP for this MAS setting is

structurally motivated rather than incidental. ECR-to-MIP modeling is a procedural task with a strict dependency chain (data extraction, formulation, coding, verification, and interpretation) whose task structure matches the conditions under which SOP is theorized to deliver value: decomposable into specialized sub-tasks, dependent on ordered handoffs, and demanding reproducible, auditable quality. Moreover, multi-agent collaboration requires an explicit coordination mechanism to avoid the inconsistency and uncontrollability of unconstrained interaction, and SOP provides this governance through a fixed division of labor, an order-constrained activation rule, standardized structured outputs, and exception backtracking. This rationale is empirically supported by Hong et al. [29], who demonstrated the efficacy of SOP in MAS for the structurally analogous task of software development. Building on this foundation, this study adapts SOP to ECR optimization. The framework is built around a workflow orchestration mechanism and a set of agents designed for ECR optimization tasks.

3.1. Agent Definition

Given an agent set $\mathcal{A} = \{a_{\mathrm{PM}}, a_{\mathrm{DE}}, a_{\mathrm{ME}}, a_{\mathrm{PY}}, a_{\mathrm{TE}}, a_{\mathrm{BA}}\}$, comprising a Project Manager Agent (a_{PM}), a Data Engineer Agent (a_{DE}), an SCM Modeling Expert Agent (a_{ME}), a Python Developer Agent (a_{PY}), a Testing Engineer Agent (a_{TE}), and a Maritime Business Analyst Agent (a_{BA}).

1) **Project Manager Agent ($a_{\mathbf{PM}}$):** Responsible for executing state transitions and managing task allocation. It initiates the appropriate agents, assigns tasks according to the workflow plan, and monitors progress to ensure reliable task completion.

2) **Data Engineer Agent ($a_{\mathbf{DE}}$):** Its role is data engineering, transforming raw user input (e.g., natural language requirements and data sources including but not limited to CSV/JSON) into structured parameters required for mathematical modeling. It records data processing activities and quality concerns, ensuring that the resulting parameters are directly usable by the Modeling Expert Agent (a_{ME}).

3) **Modeling Expert Agent ($a_{\mathbf{ME}}$):** Converts the natural language problem description into a computable mathematical model.

4) **Python Developer Agent ($a_{\mathbf{PY}}$):** It obtains the optimization model from the Modeling Expert Agent (a_{ME}) and the parameters from the Data Engineer Agent (a_{DE}). Based on this information, it composes a single, robust, and

efficient Python script that employs an optimization solver to address the specified optimization problem. The resultant output from the Python Developer Agent (a_{PY}) consists of a code block that includes a comprehensive, executable function, as well as any required import statements.

5) **Testing Engineer Agent ($a_{\mathbf{TE}}$):** When selected by the Project Manager Agent (a_{PM}), the framework executes the Python Developer Agent (a_{PY})'s solver code with the actual input data and supplies the raw execution outcome (solver status, objective value, runtime logs) to the Testing Engineer Agent (a_{TE}). The Testing Engineer Agent (a_{TE}) then interprets this outcome, classifies errors into code-level, model-level, or data-level categories, and generates a diagnostic report.

6) **Business Analyst Agent ($a_{\mathbf{BA}}$):** Container supply chain optimization often involves complex MIP problems, which are traditionally solved by automatic solvers such as Gurobi, CPLEX, and OR-Tools. However, business stakeholders often struggle to understand the reasoning behind the optimization results, which leads to repeated communication with experts and a slow decision-making process that depends on manual interpretation. Motivated by prior work [15], this paper defines the Business Analyst Agent (a_{BA}).

Each agent $a \in \mathcal{A}$ can be formally defined as a tuple: $a = (f_a, L_a, \mathcal{T}_a, t_a, \mathcal{H}_a)$, where f_a represents the core functional module of the agent, which processes one or more inputs to produce an output; L_a denotes the large language model selected for the agent (e.g., GPT-4, Claude 3, etc.); $\mathcal{T}_a$ refers to the specific prompt template designed for the agent's role; t_a indicates the temperature parameter utilized during LLM inference; and $\mathcal{H}_a$ represents a set of supplementary hyperparameters, such as `top_p` (nucleus sampling threshold) and `max_tokens` (maximum output length).

The operational boundary of each agent is delineated by its prompt template $\mathcal{T}_a$, which specifies the agent's permitted inputs, expected output format, and task scope, confining each agent to a well-defined sub-task.

3.2. Inter-Agent Communication Protocol

The six agents do not communicate peer-to-peer but coordinate exclusively through a shared message pool C, which serves as a blackboard-style communication medium. The pool C is an ordered log in which each entry records the producing agent together with its structured output. Two primitives govern access: on activation, an agent reads the current state of C as context, and after computing its result it appends a new entry to C. No agent directly invokes or modifies another

agent's internal state, which enforces a strict separation of concerns and leaves every intermediate artifact available for inspection and verification.

The output format of each message is dictated by the producing agent's role. The Data Engineer Agent (a_{DE}) emits structured JSON specifying the extracted parameters, and the Modeling Expert Agent (a_{ME}) emits a markdown-formatted model specification (parameters, variables, objective, constraints); the Python Developer Agent (a_{PY}) emits an executable code block; the Testing Engineer Agent (a_{TE}) emits a diagnostic report that classifies errors into code-level, model-level, or data-level categories; and the Business Analyst Agent (a_{BA}) emits a business-interpretable report. This shared, structured representation lets each downstream agent parse its required inputs directly from C without private handshakes. The order in which agents are activated, and hence the order in which messages accumulate in C, is constrained by the sub-sequence rule defined in the workflow orchestration (Section 3.3).

3.3. Orchestration of Workflow Based on SOP-MAS

This framework supports task execution through structured collaboration among specialized agents. The sequence of agent interactions under SOP is illustrated in Figure 1. Central to this orchestration is the *sub-sequence rule* (the order-constraint component of the SOP governance): agents are activated in the fixed role order $a_{\mathrm{DE}} \rightarrow a_{\mathrm{ME}} \rightarrow a_{\mathrm{PY}} \rightarrow a_{\mathrm{TE}} \rightarrow a_{\mathrm{BA}}$, and once selected, only downstream roles remain selectable. This order traces the linear dependency chain of MIP modeling, in which each agent's output is the next one's input (parameters $\rightarrow$ model $\rightarrow$ code $\rightarrow$ validation $\rightarrow$ interpretation), so out-of-order selection would break the chain. The workflow comprises the following steps:

Step 1: Parameter Setting and Initialization. Characterize the natural language description of the input problem as P, the maximum number of agents to be selected per forward pass as K, and the predetermined set of agents as $\mathcal{A} = \{a_{\mathrm{PM}}, a_{\mathrm{DE}}, a_{\mathrm{ME}}, a_{\mathrm{PY}}, a_{\mathrm{TE}}, a_{\mathrm{BA}}\}$. Proceed with the initialization of a message pool C, alongside the establishment of a stack H to record the selected agents, adhering to a last-in-first-out (LIFO) principle to facilitate exception backtracking. Throughout the algorithm, the agent subscript denotes a fixed agent identity, while k ($k = 1, 2, \ldots, K$) denotes the step index within a single forward pass.

Step 2: Agent Selection. While Hong et al. [29] also utilizes the SOP concept, it employs a sequential approach to agent selection, whereas SOP-MAS selects agents under an order-constrained sub-sequence rule. Consider the executable agent set, excluding a_{PM} itself, as the initial selectable set $\mathcal{A}_1 = \{a_{\mathrm{DE}}, a_{\mathrm{ME}}, a_{\mathrm{PY}}, a_{\mathrm{TE}}, a_{\mathrm{BA}}\}$ of a_{PM}. . Denote the output at step k as y_k, the set of selectable agents at step k as

$\mathcal{A}_k$, and the state as $s_k = (P, C, K-k, \mathcal{A}_k)$. The a_{PM} selection function is defined as:

$$a_k = \pi_{\text{PM}}(s_k), \quad a_k \in \mathcal{A}_k \tag{1}$$

where s_k ~~denotes the current state comprising the problem description, message pool history, remaining selection quota, and the current selectable set $\mathcal{A}_k$~~is the current state, with P denoting the problem description, C the message pool history, $K-k$ the remaining selection quota, and $\mathcal{A}_k$ the current selectable agent set; π_{PM} denotes the a_{PM} selection function; $\mathcal{A}_k \subseteq \mathcal{A}$ denotes the set of agents available for selection at step k; and a_k denotes the selected agent at step k. Here, π_{PM} is an LLM-based decision function that maps the current state s_k to a selected agent $a_k \in \mathcal{A}_k$. The strategy for selecting an agent can be transformed into the design of the prompt template $\mathcal{T}_{\text{PM}}$, with the aim of attaining an optimal policy through prompt engineering.

Step 3: Action Execution. According to the agent a_k selected in the previous step, the action f_k is to be executed. The output y_k of agent a_k is obtained as

$$y_k = a_k(P, C) \tag{2}$$

where the subscript k in y_k records the step position within the current forward pass, which is in general distinct from the agent's descriptive subscript (e.g., a_{ME}, a_{PY}).

Step 4: Update Shared Message Pool. The shared message pool C is implemented as a dictionary, wherein each key serves as a unique identifier, and the associated values consist of response messages generated by agents. During each inference step, the message pool is augmented by incorporating the current output:

$$C := C \cup \{y_k\}. \tag{3}$$

The sequence of remaining selectable agents is updated in accordance with the subsequence rule: let $\mathcal{A}^{\uparrow} = (a_{\text{DE}}, a_{\text{ME}}, a_{\text{PY}}, a_{\text{TE}}, a_{\text{BA}})$ denote the agents ordered by their fixed role sequence, $\mathcal{A}^{\uparrow}_{>a_k}$ denote the agents appearing after a_k in $\mathcal{A}^{\uparrow}$, and define the selectable set for the next step as

$$\mathcal{A}_{k+1} := \mathcal{A}^{\uparrow}_{>a_k}. \tag{4}$$

Finally, push a_k onto the stack H.

Step 5: Iterative Modeling. The process of Steps 2–4 repeats for $k = 1, 2, \ldots, K$. Upon completing all K steps, the message pool $C = \{y_1, y_2, \ldots, y_K\}$ captures the full collaboration chain, and the stack $H = (a_1, a_2, \ldots, a_K)$ records the selection sequence for potential backtracking. The framework then proceeds to Step 6.

Step 6: Failure Detection. After the K-step iterative modeling process, the Failure Detection stage begins. The framework executes the latest Python Developer Agent (a_{PY}) solver code to obtain the execution outcome. If the Testing Engineer Agent (a_{TE}) was selected during the forward pass, it interprets the execution outcome, classifies errors into code-level, model-level, or data-level categories, and generates a structured diagnostic report. The correctness gate is then applied as follows:

- *For benchmark instances:* The solution is deemed correct only if (i) the solver reports an Optimal status, and (ii) the objective value matches the ground-truth optimum.
- *For real-world ECR cases:* The solution must satisfy all constraints as programmatically checked during execution and pass an SCM human expert sanity check.

If the Failure Detection stage succeeds, the workflow proceeds directly to Step 8. Otherwise, if `is_backward` = `True`, the Project Manager Agent (a_{PM}) triggers Step 7 for exception backtracking; if `is_backward` = `False`, the workflow skips backtracking and proceeds directly to Step 8.

Step 7: Exception Backtracking. When the execution or verification stage fails and `is_backward` = `True`, trigger the backtracking mechanism. The Project Manager Agent (a_{PM}) pops the stack H in reverse order and queries previously selected agents one by one to determine whether their outputs caused the failure. Extract the top agent a_k from the stack H and evaluate whether its output y_k was the source of the exception. Revert the message pool C to its state prior to the execution of a_k, by removing the outputs of a_k and all subsequent agents: update $C := C \setminus \{y_k, y_{k+1}, \ldots\}$. Restore the selectable set to the corresponding $\mathcal{A}_k$, i.e., the feasible agent subset immediately before selecting a_k. Obtain feedback error information through

$$(\delta_k, \xi_k) \leftarrow a_k\,(C)\,, \tag{5}$$

where $\delta_k \in \{0, 1\}$ is an indicator variable for the queried agent's responsibility judgment: $\delta_k = 1$ means that agent a_k claims responsibility for the failure and provides a revised output, whereas $\delta_k = 0$ means that agent a_k does not claim responsibility and the exception is propagated upward to the next agent popped from the stack. The accompanying ξ_k represents a natural language error description for localization or revision. If the Testing Engineer Agent (a_{TE}) was selected during this iterative modeling process, ξ_k may include its structured execution report; otherwise,

the diagnosis relies on the failure signal returned by the execution and verification stage. Because the forward workflow follows the monotonic order specified by $\mathcal{A}^{\uparrow}$, implementation-level defects are typically examined first at the Python Developer Agent (a_{PY}), whereas failures that persist beyond code correction are propagated further to the Modeling Expert Agent (a_{ME}) for model-level revision. When $\delta_k = 1$, the framework updates the message pool with the revised output from a_k and resumes forward modeling from the restored selectable set. When $\delta_k = 0$, the Project Manager Agent (a_{PM}) continues the LIFO traversal until an agent claims responsibility or the stack is exhausted. In particular, when a failure originates from multiple agents, the framework resolves them over successive backtracking rounds: each round in which some agent claims responsibility ($\delta_k = 1$) produces a revised output and a fresh forward pass, and any error that remains triggers the next round, popping the next agent in reverse order, until all culprits are corrected or the collaboration budget K is exhausted.

Step 8: Termination Condition. The algorithm concludes its execution and yields the solution.

The proposed SOP-MAS framework is designed as an expert-in-the-loop decision support system. In real-world container logistics operations, particularly in complex, high-stakes scenarios like ECR, the ultimate responsibility for model validity, constraint fidelity, and operational feasibility rests with domain experts (e.g., logistics planners, operations researchers, or port analysts).

3.4. Computational Complexity Analysis

We complement the empirical efficiency study (Section 4.4) with a formal analysis of the orchestration layer's computational complexity. The end-to-end wall-clock cost of one SOP-MAS run decomposes into three disjoint components,

$$T_{\mathrm{total}} = T_{\mathrm{orch}} + T_{\mathrm{LLM}} + T_{\mathrm{solver}}, \tag{6}$$

where T_{orch} is the orchestration overhead (all non-LLM framework operations), T_{LLM} is the aggregate LLM inference cost, and T_{solver} is the cost of executing the generated code with the MIP solver.

The orchestration overhead T_{orch} comprises the non-LLM operations performed by the framework. Appending an agent's output to the message pool is $O(1)$, contributing $O(K)$ over a forward pass. Constructing the selectable set $\mathcal{A}_k$ under the sub-sequence rule scans at most $|\mathcal{A}|$ agents per step, contributing $O(K \cdot |\mathcal{A}|)$. Stack push/pop operations for exception backtracking are $O(1)$ each, bounded by $O(K)$ across a run. Hence $T_{\mathrm{orch}} = O(K \cdot |\mathcal{A}|)$, which, for the fixed agent set $|\mathcal{A}| = 6$

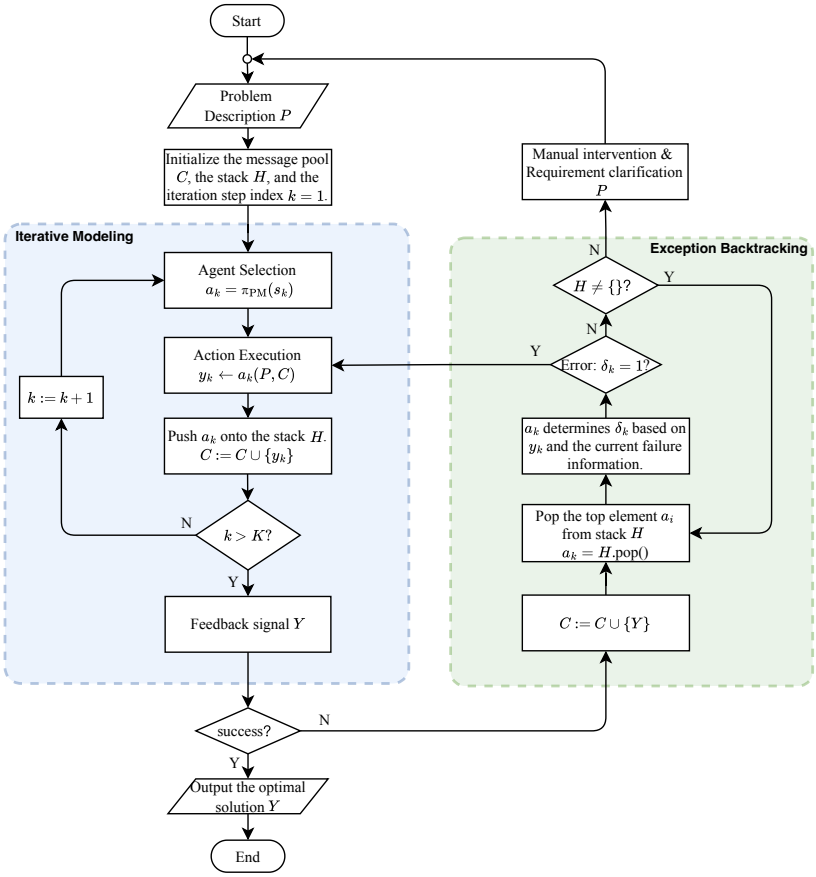


Figure 1: SOP-MAS flowchart

and a small collaboration budget (typically $K \leq 5$), reduces to a constant that is independent of the problem scale (the number of decision variables and constraints).

The remaining two terms are external to the orchestration logic. T_{LLM} scales with the number of invocations, $O(K)$, and with the context length at each call, as the message pool grows across steps; as characterized empirically in Section 4.4, it is the dominant component of the end-to-end runtime. T_{solver} depends on the problem scale but is incurred a small number of times bounded by the collaboration budget K and resides entirely in the solver, whose scalability the operations research community has extensively studied. The orchestration layer, which is the algorithmic contribution of this work, therefore adds a scale-independent, constant overhead. In summary,

$T_{\text{orch}} = O(K \cdot |\mathcal{A}|)$ reduces to $O(1)$ with respect to the MIP problem scale, and the end-to-end runtime is governed by T_{LLM} rather than by orchestration.

3.5. Mechanisms for Hallucination Mitigation

Within the framework of the SOP-MAS methodology, a systematic strategy is adopted to mitigate the pervasive issue of hallucinations in LLMs. This strategy encompasses the integration of prompt engineering, SOP, and a mechanism for exception backtracking and correction.

1) **Meticulously Designed Prompt Engineering.** To ensure each agent accurately comprehends its role and tasks, and produces outputs that are standardized, this study designs refined prompt templates for each agent. These templates adhere to a unified structure, encompassing key elements such as role definition, principal tasks, input/output specifications, and constraints. By restricting the generation scope and steering the reasoning process through prompt engineering, the LLMs are constrained to produce output in role-specific structured formats (JSON for parameter and diagnostic outputs, a markdown model specification, or executable code templates), thereby enforcing structured thinking and minimizing the production of redundant or irrelevant information. Owing to space limitations, this paper will not display all prompts in full but will place their core design logic along with a typical example (e.g., for the Modeling Expert Agent (a_{ME})) in Appendix A.

2) **Constraining Roles by the SOP.** The SOP mitigates risks by clearly defining the responsibilities of each agent and by establishing quality standards for intermediate outputs. The use of role-specific structured formats (JSON and markdown specifications) as the inter-agent communication protocol restricts each agent's latitude for unstructured "free-form" output, requiring their responses to adhere to specific formats and standards, thereby diminishing the likelihood of generating nonsensical or irrelevant outputs, termed hallucinations. Furthermore, the logical consistency and mathematical correctness of the generated models and code across different agents function as a mechanism for cross-verification.

3) **Exception Backtracking and Correction Mechanism.** Upon the occurrence of an execution or verification failure, the framework generates the corresponding failure information for backtracking. If the Testing Engineer Agent (a_{TE}) was selected during the iterative modeling process, its structured execution report stored in the shared message pool—capturing the solver log,

error trace, and root-cause analysis—serves as the primary diagnostic artifact; otherwise, backtracking relies only on the failure information returned by the framework's own execution and verification stage. Concurrently, a stack pop operation is performed on H, sequentially popping the topmost agents to check whether the error was caused by the current agent's output. This backtracking mechanism allows for pinpointing the source of errors and facilitates corrective actions, significantly enhancing the robustness of the SOP-MAS framework. By enabling agents to examine and potentially amend their outputs in response to subsequent execution feedback, the system is enabled to autonomously rectify and recover from errors attributable to LLM inaccuracies.

3.6. Illustrative Example

To demonstrate the end-to-end capability of the proposed SOP-MAS framework, we apply it to a single-period, three-port ECR instance. Ports A and B hold surplus containers (100 and 50 TEU, respectively) while port C demands 150 TEU; the unit transportation costs c_{ij} range from 2 to 10 ¥/TEU, and x_{ij} denotes the number of empty containers repositioned from port i to port j. The SOP-MAS is configured with K=5 collaborating agents, backtracking enabled, DeepSeek-V3-0324 as the backbone LLM, and Gurobi as the solver. The complete workflow is illustrated in Figure 2. The Project Manager Agent (a_{PM}) orchestrates the five agents in sequence. First, the Data Engineer Agent (a_{DE}) parses the natural-language problem description and extracts structured parameters: the supply vector S=[100, 50, 0], the demand vector D=[0, 0, 150], and the cost matrix c_{ij}. Second, the Modeling Expert Agent (a_{ME}) formulates the LP: $\min \sum_{i,j} c_{ij}x_{ij}$ subject to flow-balance constraints $\sum_j x_{ij} - \sum_j x_{ji} = S_i - D_i$ for each port i and non-negativity $x_{ij} \geq 0$, but inadvertently adds two extra constraints x_{AB}=0 and x_{BA}=0 that forbid transshipment between the surplus ports. Third, the Python Developer Agent (a_{PY}) generates executable solver code faithfully implementing this model. Fourth, because the Testing Engineer Agent (a_{TE}) is selected in this K=5 example, the framework executes the solver code; Gurobi solves the resulting LP in 2 iterations, yielding x_{AC}=100, x_{BC}=50 and a total repositioning cost of ¥1150. Fifth, the Business Analyst Agent (a_{BA}) translates this solution into a business-analysis report. After the forward pass, the framework enters its unified verification stage and detects a mismatch between the solver output and the expected optimum of ¥650; because `is_backward=True`, it initiates exception backtracking.

The framework pops the agent stack in LIFO order and queries each agent via its `backward_step`, which receives the error information together with the agent's own prior output and returns a structured JSON judgment. The Python Developer

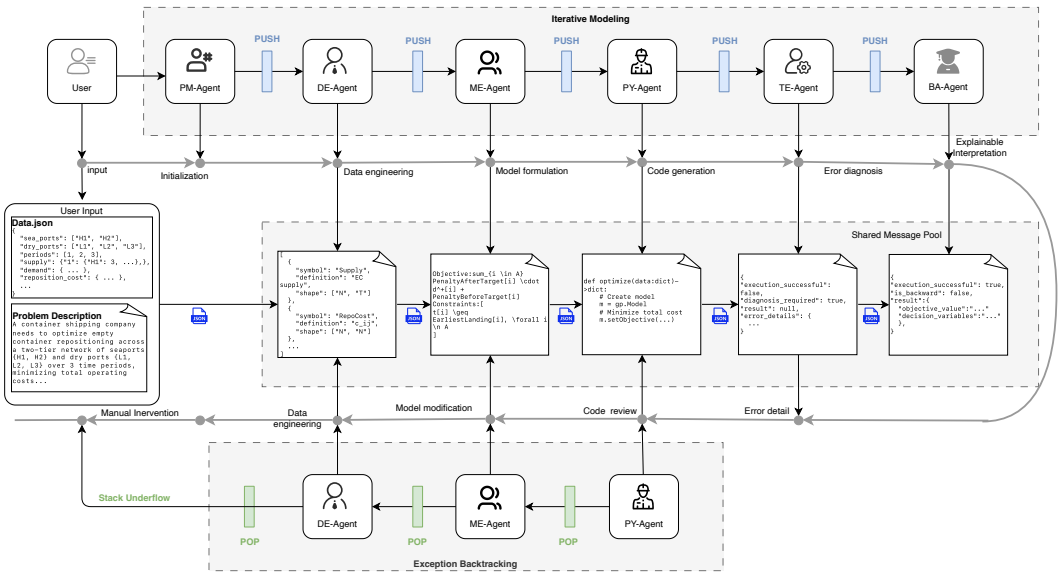


Figure 2: End-to-end SOP-MAS workflow on an ECR instance (3 ports, T=1): from natural-language description to optimal repositioning plan

Agent (a_{PY}) returns δ_3=0 with ξ_3 stating that it does not claim responsibility because the code faithfully implements the received formulation, so the exception propagates upward. The Modeling Expert Agent (a_{ME}), upon seeing the verification failure, returns δ_2=1 with ξ_2 identifying the erroneous constraints x_{AB}=0 and x_{BA}=0 and providing a corrected formulation. The framework pushes the refined model back onto the message pool, resumes forward modeling from the Modeling Expert Agent (a_{ME}) onward, and the corrected formulation yields the optimal solution (x_{AB}=100, x_{BC}=150, all other flows zero) with a total cost of ¥650. The Business Analyst Agent (a_{BA}) then produces a revised business-analysis report, correctly explaining why the low-cost A–B and B–C arcs satisfy port C's demand through Port B as the transfer hub. This example demonstrates that $\delta_k = 1$ denotes responsibility and revision, whereas $\delta_k = 0$ denotes upward propagation to the next popped agent.

This end-to-end workflow—(1) natural-language description, (2) structured modeling, (3) solution-code generation and execution, (4) interpretable results—largely parallels the conventional MIP modeling process, with the MAS assigning individual agents to authentic enterprise roles. The backtracking example further demonstrates that the LIFO reverse traversal examines code-level defects before formulation-level ones, and that each correction round consumes at most one remaining collaboration budget. Beyond this, the SOP framework offers distinctive advantages in task de-

composition, agent scheduling, automated error recovery, and iterative multi-round collaboration.

4. System Implementation and Experimental Analysis

The effectiveness of the SOP-MAS method is validated against baseline approaches using publicly available datasets. To identify the optimal hyperparameter configuration, sensitivity analyses are conducted across problem instances of varying scales. All experiments are implemented in Python 3.11.9 on a local machine running Windows 10 Professional, equipped with an Intel i5-4210U CPU and 32 GB of DDR3 memory. The local machine, is exclusively responsible for orchestrating the multi-agent workflow, executing the generated solver code within a sandbox, and managing the shared message pool, while the DeepSeek-V3-0324 model is accessed via a cloud API.

4.1. Comparison with Baseline Methods and Effectiveness Analysis

To validate the effectiveness of SOP-MAS, a total of 269 instances were selected from the ComplexOR and NLP4LP benchmark datasets. The ComplexOR dataset Xiao et al. [33] is explicitly crafted to provide benchmark tests for complex OR problems, comprising 37 multi-source, heterogeneous scenarios spanning supply chain optimization, scheduling, and warehousing logistics. Each scenario is expert-annotated, including both model formulations and test cases to verify the correctness of generated code. The NLP4LP dataset comprises LP and MIP scenarios, with problem descriptions distinguished by lengthy textual narratives, implicit constraints, and complex data structures (e.g., multi-resource allocation and production scheduling), necessitating domain expertise for accurate mathematical modeling. To strengthen evaluation fidelity, we further refined the annotation of all instances by structuring them into four standardized components:

1) a natural-language problem description (capturing business intent, constraints, and objectives),

2) structured modeling metadata (including variable definitions, parameter schemas, and constraint templates),

3) sample input data (e.g., CSV/JSON instances for model instantiation), and

4) reference optimization outcomes (optimal/near-optimal solutions and solver logs for correctness verification).

This design establishes a closed-loop evaluation framework: SOP-MAS ingests 1) to 3) and is assessed on its ability to reproduce 4) without manual intervention. This structure not only enables rigorous quantitative evaluation but also supports qualitative analysis of modeling fidelity and interpretability, key criteria for real-world deployment.

The performance of SOP-MAS on these instances is evaluated by comparing it against baseline methods on benchmark cases, specifically including the Standard Prompting Method (SPM), CoT, CoE [33], and OptiMUS [32]. We adopt DeepSeek-V3-0324 as the backbone LLM for all methods, given its leading position among open-weight models. Crucially, this work does not aim to benchmark LLM capabilities; rather, it seeks to evaluate a novel workflow orchestration paradigm—SOP-MAS. Therefore, we do not employ other LLMs, as is consistent with foundational multi-agent frameworks such as AutoGen [28] and MetaGPT [29]. All baselines were evaluated under the same DeepSeek-V3-0324 backbone and identical hardware conditions to ensure a fair comparison.

Similarly, Gurobi is employed as the solver, a high-performance optimization engine widely used for MIP/LP in academia and industry. The Python Developer Agent (a_{PY}) generates solver-agnostic Python code; adapting the same prompt template to other solvers such as OR-Tools or CPLEX requires only minimal syntactic substitution (e.g., changing the solver creation call), confirming the framework's portability. Thus, validation under different solvers would assess solver characteristics rather than the efficacy of SOP-MAS itself, which lies beyond the scope of this study.

Following the metrics proposed in Xiao et al. [33], the evaluation was performed utilizing Compile-Time Errors (CER), Run-Time Errors (RER), and Accuracy Rate (AR). For each test instance $i \in \{1, \ldots, N\}$, let $c_i \in \{0, 1\}$ indicate successful compilation, $r_i \in \{0, 1\}$ indicate execution without runtime errors, s_i denote the solver status, z_i denote the objective value returned by the generated code, and $\hat{z}_i$ denote the ground-truth optimal objective value. CER and RER are defined as

$$\mathrm{CER} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(c_i = 0). \tag{7}$$

$$\mathrm{RER} = \frac{\sum_{i=1}^{N} \mathbf{1}(c_i = 1,\ r_i = 0)}{\sum_{i=1}^{N} \mathbf{1}(c_i = 1)}. \tag{8}$$

An instance is considered correctly solved only when the generated code compiles, executes without error, attains an optimal solver status, and yields an objective value

Table 1: Comparison of SOP-MAS and baseline algorithms

Baseline	CER	RER	AR
Standard Prompting Method (SPM)	7.67%	11.81%	48.08%
Chain-of-Thought (CoT)	7.32%	10.83%	52.26%
Chain-of-Experts (CoE)	4.18%	5.77%	62.37%
OptiMUS	3.14%	4.51%	77.35%
SOP-MAS	1.39%	2.91%	86.59%

within tolerance of the ground truth, i.e.,

$$a_i = \mathbf{1}\left(c_i = 1,\ r_i = 1,\ s_i = \text{Optimal},\ \frac{|z_i - \hat{z}_i|}{\max(1, |\hat{z}_i|)} \leq \varepsilon\right). \tag{9}$$

The accuracy rate is then defined as

$$\text{AR} = \frac{1}{N}\sum_{i=1}^{N} a_i. \tag{10}$$

In this study, $\varepsilon = 10^{-6}$. Each algorithm was executed five times with $K = 5$. Mean results are summarized in Table 1.

The findings suggest that the SPM methodology exhibits the lowest accuracy among all evaluated approaches, mainly due to its black-box reasoning which concentrates solely on the final solution, while neglecting the model's intermediate reasoning steps. This deficiency may result in imprecise outputs when tackling complex problems. In contrast, the CoT method, by employing stepwise reasoning, enhances the model's interpretability and displays better performance on complex instances, realizing an increase in accuracy from 48.08% to 52.26% (a relative improvement of 8.69%) over SPM. Both the CoE and OptiMUS methods reasonably capitalize on multi-agent collaboration to address complex optimization problems effectively. Among all baseline approaches, the proposed SOP-MAS method achieves the highest accuracy.

Notably, the SOP-MAS framework achieves an accuracy rate of 86.59 percent, which falls short of perfect automation. This outcome is neither unexpected nor undesirable; rather, it reflects the intrinsic complexity of real-world operational modeling. In practice, even seasoned SCM experts seldom produce a fully correct MIP model on the first attempt. Instead, model development routinely proceeds through multiple cycles of refinement, guided by feasibility diagnostics, sensitivity testing,

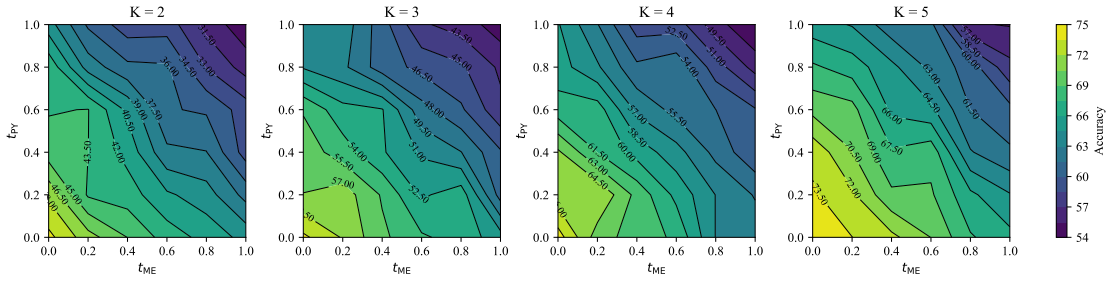


Figure 3: Faceted contour plot of parameter sensitivity analysis

and iterative feedback from domain stakeholders. Consistent with this reality, SOP-MAS is deliberately designed as a human-in-the-loop decision support tool rather than a fully autonomous optimizer. Its primary objective is to accelerate the expert modeling workflow, which in practice often spans hours or even weeks for complex real-world problems, by automating routine tasks such as data parsing, constraint drafting, and code generation, while simultaneously surfacing potential inconsistencies or violations for expert review. Final validation, business-rule checking, and strategic acceptance remain the responsibility of human experts. In doing so, the framework shifts human effort away from mechanical implementation toward high-value activities such as model validation, business logic verification, and strategic refinement.

4.2. Sensitivity Analysis of LLM Parameters

Within the SOP-MAS framework, the maximum collaboration count K of the Project Manager Agent (a_{PM}) governs the depth and breadth of the workflow by modulating the number of agent invocations. Among the six agents, the temperatures of the Modeling Expert Agent (a_{ME}) and the Python Developer Agent (a_{PY}), denoted as t_{ME} and t_{PY}, are the primary determinants of solution accuracy, because these two agents are directly responsible for mathematical model construction and executable code generation, respectively.

In order to systematically evaluate the effects of parameter configurations on the performance of multi-agent collaboration, a three-dimensional sensitivity analysis framework was developed using the benchmark instance set. The exception backtracking mechanism was deactivated (`is_backward` = `False`) to eliminate any confounding variables. A grid search was performed over the parameters $K \in [2, 5]$ and $t_{\mathrm{ME}}, t_{\mathrm{PY}} \in \{0.2, 0.4, 0.8, 1.0\}$, utilizing AR as the evaluation metric to identify the optimal parameter combination. The results are shown in Figure 3.

The Project Manager Agent (a_{PM}) orchestrates its constituent agents in accor-

dance with the predefined maximum collaboration threshold K throughout the iterative modeling process. This organization consequently influences the comprehensive scope and extent of the workflow. In instances where K is minimal, the system depends on a limited number of agents to facilitate rapid decision-making, which is appropriate for tasks requiring immediate attention. Conversely, an augmentation in K allows for the incorporation of additional domain-knowledge verification steps, rendering it more suitable for highly complex business scenarios. Notably, as K attains the value of 5, there is a notable enhancement in accuracy, indicating that multi-agent collaboration fosters specialization, with each agent concentrating on a distinct domain.

In contrast, an increase in the parameters t_{ME} and t_{PY} results in declines in accuracy. For both the Modeling Expert Agent (a_{ME}) and the Python Developer Agent (a_{PY}), elevated temperatures within the context of mathematical modeling and programming are associated with reduced accuracy. This occurs because lower temperatures substantially limit the randomness in generated outputs, thus ensuring logical rigor and syntactical precision. This is critical for tasks requiring precise derivation, such as mathematical formulae, or strict syntax, such as programming tasks, where sustaining accuracy may slightly constrain creativity but effectively diminishes error rates and enhances development efficiency.

4.3. Ablation Study

An ablation study was conducted to assess the contribution of each agent within SOP-MAS and the effectiveness of the collaborative mechanisms. The fundamental operations of SOP-MAS in addressing optimization challenges are rooted in mathematical modeling and code execution. Consequently, this study examines system performance through the selective removal of certain agents or restricting the exception-backtracking mechanism. The experimental groups are defined as follows:

1) **Group 1** ($a_{\mathbf{ME}}, a_{\mathbf{PY}}$)**:** Solely comprises the Modeling Expert Agent (a_{ME}) and the Python Developer Agent (a_{PY}), with exception backtracking deactivated, to assess the significance of collaboration among multiple agents.

2) **Group 2** ($a_{\mathbf{PM}}, a_{\mathbf{DE}}, a_{\mathbf{ME}}, a_{\mathbf{PY}}, a_{\mathbf{TE}}$)**:** Excluding the Business Analyst Agent (a_{BA}) whilst permitting exception backtracking to examine the role of the Business Analyst Agent (a_{BA}).

3) **Group 3** ($a_{\mathbf{PM}}, a_{\mathbf{ME}}, a_{\mathbf{PY}}, a_{\mathbf{TE}}, a_{\mathbf{BA}}$)**:** The Data Engineer Agent (a_{DE}) is excluded, with exception backtracking enabled, to evaluate the significance of structured modeling information.

4) **Group 4** ($a_{\mathbf{PM}}, a_{\mathbf{DE}}, a_{\mathbf{ME}}, a_{\mathbf{PY}}, a_{\mathbf{BA}}$)**:** The Testing Engineer Agent (a_{TE}) is omitted to examine the interaction between testing and exception backtracking, while exception backtracking remains enabled.

5) **Group 5** ($a_{\mathbf{PM}}, a_{\mathbf{DE}}, a_{\mathbf{ME}}, a_{\mathbf{PY}}, a_{\mathbf{TE}}, a_{\mathbf{BA}}$)**:** All six agents are retained, while exception backtracking is disabled, in order to examine the effect of the exception-backtracking mechanism on system performance.

The entirety of SOP-MAS functions as the control group for assessing performance benchmarks. The experimental results are presented in Table 2.

Table 2: Ablation study

Group No.	CER	RER	AR
Group1 (−)	6.27%	9.80%	57.84%
Group2 (+)	1.74%	2.55%	82.58%
Group3 (+)	1.74%	3.68%	73.17%
Group4 (+)	1.44%	2.09%	84.32%
Group5 (−)	3.48%	4.53%	69.69%
Control Group (+)	1.39%	2.91%	86.59%

Note. "+" and "−" indicate that exception backtracking is activated and deactivated, respectively.

In Group 1, which consists exclusively of a_{ME} and a_{PY}, there is a decrease in accuracy from 86.59% to 57.84% compared to the control group. This decline highlights the significance of multi-agent collaboration in SOP-MAS for tackling complex optimization problems. In Group 2, the removal of the Business Analyst Agent (a_{BA}) results in an accuracy level of 82.58%, indicating that the Business Analyst Agent (a_{BA}) exerts a minimal influence on overall accuracy. This is attributed to the position of the Business Analyst Agent (a_{BA}) at the terminal part of the workflow, where its primary function is the interpretation and packaging of previously obtained results. Consequently, its absence does not compromise the accuracy of front-end modeling and problem-solving but affects solely the presentation and clarity of the final output delivered to the user. The experimental metric, Accuracy Rate (AR), inherently fails to account for the Business Analyst Agent (a_{BA})'s contribution, and this limitation should be clearly indicated. Because the Business Analyst Agent (a_{BA}) operates after solution generation, removing it has limited effect on AR even though it reduces the interpretability of the final output. Its main contribution is to translate solver outputs into interpretable business reports for end

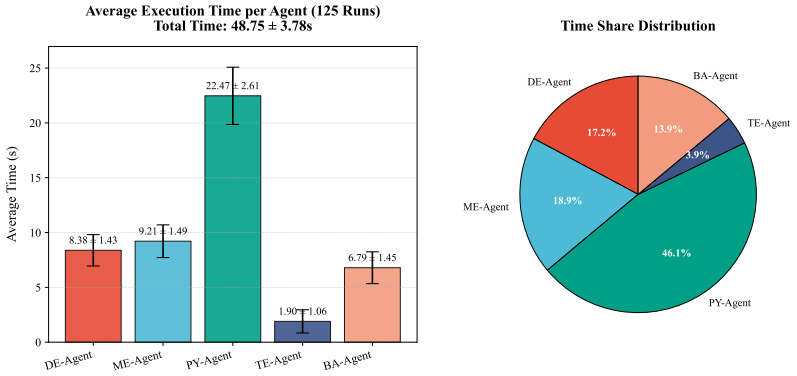


Figure 4: Average execution time and time share distribution

users, as illustrated by the three-port example in Section 3.6 and the empirical case in Section 5.3, where it explains routing decisions, decomposes total cost, and provides managerial recommendations. In Group 3, the removal of the Data Engineer Agent (a_{DE}) results in a decrease in accuracy from 86.59% to 73.17% relative to the control group, demonstrating that structured modeling information is instrumental for the Modeling Expert Agent (a_{ME}) in precisely delineating problem parameters. Group 4 achieves 84.32%, a gap of 2.27 percentage points from the 86.59% of the control group. Because code execution is always performed by the framework in Step 6 regardless of whether the Testing Engineer Agent (a_{TE}) is selected, removing the Testing Engineer Agent (a_{TE}) does not affect the baseline verification capability. The 2.27% gap reflects the marginal benefit of the Testing Engineer Agent (a_{TE})'s LLM-based diagnostic interpretation, which assists the backtracking mechanism in a subset of failure cases where the raw pass/fail signal alone is insufficient for accurate root-cause localization. This observation is consistent with the findings in Group 5, where disabling exception backtracking results in a significant decline in accuracy.

4.4. Computational Efficiency Analysis

To provide a comprehensive performance analysis, we present a detailed evaluation of the proposed SOP-MAS framework. This analysis encompasses 25 representative cases that span a range of problem scales, randomly sampled from the total pool of 269 instances. The findings are summarized in Figure 4, illustrating the roles of each agent, their average task completion times, and the variability thereof, all measured in seconds.

Our experimental results indicate that the Python Developer Agent (a_{PY}) experiences the longest average processing time among all agents, taking approximately

22.47 seconds per task with a standard deviation of 2.61 seconds. This extended duration likely reflects the complexity and variability inherent in coding processes. Conversely, the Testing Engineer Agent (a_{TE}) completes tasks in the shortest average time, requiring only about 1.90 seconds per task, with a notably small standard deviation of 1.06 seconds. This indicates a high level of consistency and efficiency in executing testing tasks.

The other agents, including the Data Engineer Agent (a_{DE}) (8.38 ± 1.43 seconds), Modeling Expert Agent (a_{ME}) (9.21 ± 1.49 seconds), and Business Analyst Agent (a_{BA}) (6.79 ± 1.45 seconds), also demonstrate efficient performance with acceptable variances, contributing to the overall robustness and practical deployability of the SOP-MAS framework.

Aggregating across all stages, the average end-to-end runtime per instance is approximately 48.75 seconds (sum of mean agent times), with an estimated total standard deviation of 2.4 seconds (propagated from individual variances). This sub-minute latency demonstrates that the framework is well-suited for tactical decision-making scenarios in port operations, where planning horizons typically span hours or days rather than real-time milliseconds. The modeling runtime of SOP-MAS is dominated by LLM API calls rather than problem scale. This scale-independence stems from the architectural separation between the abstract symbolic problem description (parameters, variables, constraints) and the concrete numerical data. Because the problem description is expressed in index notation, the model structure generated by the ME-Agent and the code template produced by the PY-Agent remain invariant to the cardinality of index sets (e.g., the number of spatial nodes or temporal stages). Consequently, the sub-minute modeling overhead of SOP-MAS stays constant as problem size increases. The actual problem scale, manifested in the number of decision variables and constraints, only affects the solver execution phase, whose scalability has been extensively validated by the OR community. This separation of concerns makes SOP-MAS suitable for industrial-scale applications where the computational bottleneck resides in the solver, not in the modeling process. Furthermore, the per-agent times reported above are end-to-end latencies that inherently bundle LLM inference, API communication, and inter-agent coordination into one measurement. Since agents execute sequentially with lightweight message-pool operations at each handoff, the multi-agent coordination overhead is negligible relative to the LLM inference time; the dominant factor is the reasoning complexity of individual agents. The reported 48.75 s should be interpreted as the time required to generate an executable initial model, rather than the full human modeling cycle, which additionally involves requirement clarification, iterative formulation revision, debugging, and stakeholder validation.

We also report prompting token consumption for each paradigm. Table 3 and Figure 5 list the mean number of LLM calls and mean total tokens per instance, together with the AR reported in Table 1.

Table 3: Prompting token overhead across paradigms (K=5, 25 NLP4LP instances, seed 42). AR is cited from Table 1.

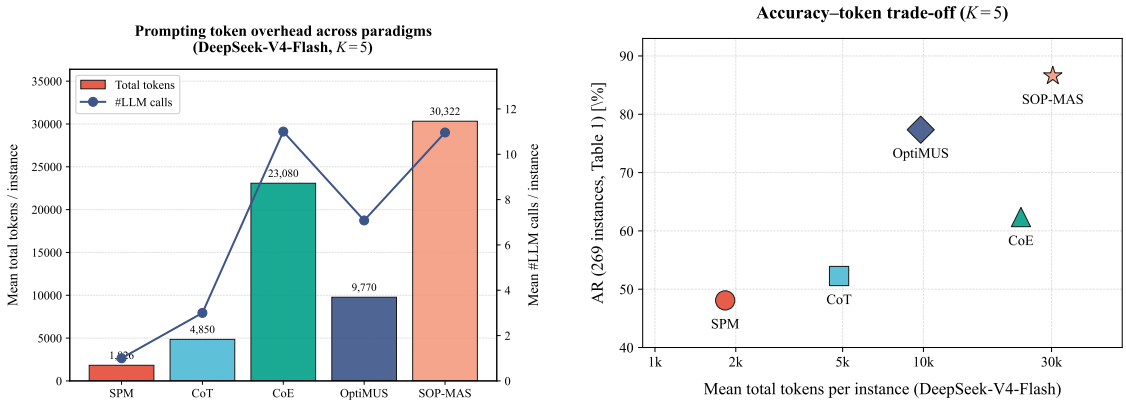
Method	LLM calls (mean)	Total tokens (mean)	AR
SPM	1.0	1,826	48.08%
CoT	3.0	4,850	52.26%
CoE	11.0	23,080	62.37%
OptiMUS	7.1	9,770	77.35%
SOP-MAS	11.0	30,322	86.59%

Prompting overhead increases monotonically with collaboration complexity, from single-shot SPM (≈1.8k tokens) through CoT, OptiMUS, and CoE to SOP-MAS (≈30k tokens), consistent with multi-agent collaboration and exception backtracking. Figure 5b shows a positive association between mean total tokens and AR: SOP-MAS consumes the largest prompting budget (≈30k tokens per instance) and achieves the highest accuracy in Table 1 (86.59% versus 48.08–77.35%). In typical ECR applications, per-instance repositioning, leasing, and holding costs exceed API expenditure by orders of magnitude.

5. Empirical Analysis

5.1. Case Setup and Data Sources

This section selects ECR as the empirical testbed because it represents a high-stakes, data-rich problem that demands both mathematical rigor and business interpretability, precisely the dual requirements our SOP-MAS framework is designed to fulfill. Unlike generic optimization benchmarks, real-world ECR involves intricate trade-offs among transportation cost, multimodal transport, and contractual obligations across multiple stakeholders, making it an ideal domain for evaluating whether SOP-MAS-driven modeling can transition from theoretical promise to industrial utility. The empirical analysis examines the Liaoning coastal port cluster in Northeast China, which comprises three seaports with complementary roles: Dalian (international transshipment hub for the northeastern economic region, encompassing Liaoning, Jilin, and Heilongjiang provinces as well as the four eastern leagues of Inner Mongolia), Yingkou (low-cost land transport, forming a dual-core structure



(a) Token overhead

(b) Accuracy–token trade-off

Figure 5: Prompting token overhead across paradigms (K=5): (a) mean total tokens (bars) and number of LLM calls (line) per instance; (b) AR (cited from Table 1) versus mean total tokens, with SOP-MAS at the upper right.

with Dalian), and Dandong (logistics gateway for eastern Northeast China). These ports share overlapping hinterlands that form key nodes for sea–rail intermodal transportation (Figure 6).

A planning horizon of five 7-day periods is adopted. Following Cai et al. [11], the freight rates for empty container transportation by maritime, road, and rail are 0.8, 4.5, and 1.8 CNY/TEU/km, respectively; the leasing cost at seaports is 4,000 CNY/TEU; the holding cost is 10 CNY/TEU/day at seaports and 5 CNY/TEU/day at dry ports; the supply and demand of empty containers at each node per period follow a normal distribution. Inter-node distances and sea-repositioning transit times are standard road and sea values provided in the public SOP-MAS repository, and the per-node empty-container demand and supply distributions are reported in Appendix B.

5.2. Problem Description

The ECR network is modeled as a directed graph $G = (\mathcal{N}, E)$, where $\mathcal{N} = \mathcal{P} \cup \mathcal{L}$ partitions nodes into the seaport set $\mathcal{P}$ and the dry port set $\mathcal{L}$, and $T = \{1, \ldots, |T|\}$ indexes the planning periods. The edge set E represents repositioning routes among seaports, between seaports and dry ports, and among dry ports. A binary parameter $\lambda_{ij} = 1$ indicates that dry port j belongs to the hinterland of seaport i; when two dry ports $j_1, j_2 \in \mathcal{L}$ share the same affiliated seaport ($\lambda_{ij_1} = \lambda_{ij_2} = 1$), direct inter-dry-port repositioning is permitted. The empty container supply Q_i^t at each node derives from inbound laden containers in the preceding period, while outbound laden

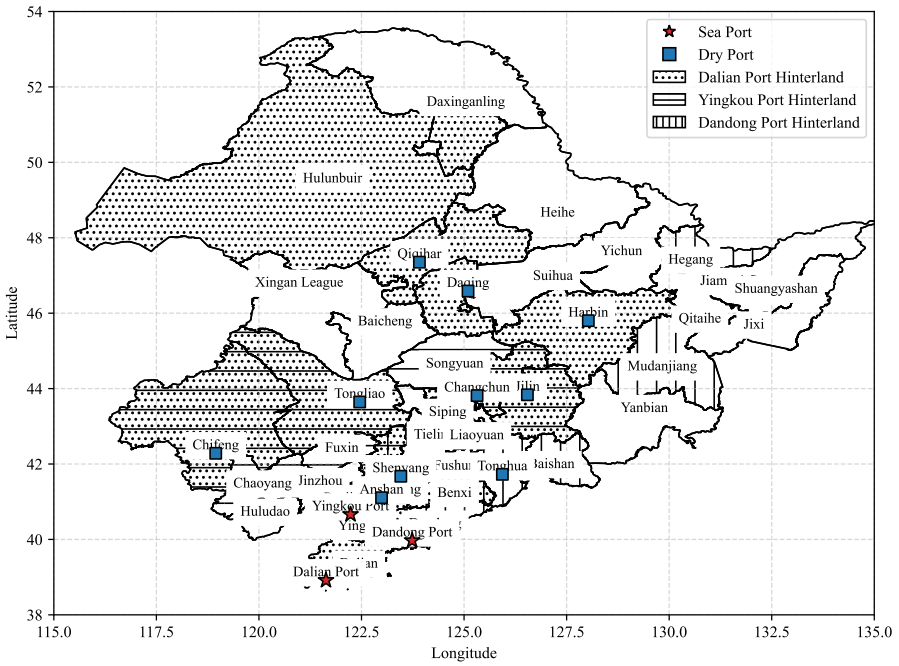


Figure 6: Port hinterland map

containers constitute the demand D_i^t. Each node has a maximum storage capacity $U_i^{\max}$ and a per-period holding cost c_i^{hold}. Shortages are met by repositioning from other nodes at unit cost c_{ijm}^{repos} via transport mode $m \in \mathcal{M}$ (subject to capacity $F_{ijm}^{\max}$), or by leasing from external providers at cost c_i^{rent}. The objective is to minimize the aggregate repositioning, storage, and leasing costs over the planning horizon, subject to the following assumptions:

1) Empty containers may be repositioned among seaports, between seaports and dry ports, and among dry ports.

2) Both seaports and dry ports can store empty containers, subject to maximum capacity.

3) Container leasing is available in virtually unlimited quantity at each node.

The complete mathematical formulation is provided in Appendix C.

5.3. Results Analysis

Based on the ECR problem setting described in Section 5.2 and the data presented in Section 5.1, the SOP-MAS framework was employed for end-to-end optimization.

Under the sea–land coordination scenario (Scenario 1), wherein hinterland lateral repositioning is permitted, the optimal total cost over the five-period planning horizon is CNY 1,190,336, comprising repositioning costs of CNY 763,966, holding costs of CNY 30,870, and leasing costs of CNY 395,500, as summarized in Table 4.

To evaluate the benefits of hinterland lateral repositioning, a comparative analysis was conducted under the scenario without inter-dry-port exchange (Scenario 2), as illustrated in Figure 7. As shown in Table 4, the total cost increases from CNY 1,190,336 to CNY 1,355,388. This increase is primarily driven by the repositioning costs, which rise from CNY 763,966 to CNY 925,238 (a 17.43% saving under Scenario 1). Within the repositioning category, the sea–land repositioning cost surges from CNY 594,952 to CNY 864,910 (a 31.21% saving under Scenario 1), as the network must compensate for the loss of hinterland lateral flows (CNY 110,718 reduced to zero) by routing more containers through the sea–dry corridors. Furthermore, the exacerbated difficulty in empty container flows leads to a greater number of containers failing to arrive at shortage seaports in a timely manner, which in turn increases the leasing demand at the affected seaports from CNY 395,500 to CNY 406,000 (a 2.59% saving under Scenario 1). Overall, enabling hinterland lateral repositioning yields a 12.18% reduction in total operational costs. Because the two scenarios differ solely in whether hinterland lateral repositioning is enabled, this entire reduction is the marginal effect attributable to the lateral land route: the CNY 110,718 spent on hinterland lateral flows avoids CNY 269,958 in sea–land repositioning, CNY 10,500 in leasing, and CNY 2,032 in seaport feeder repositioning, yielding a net saving of CNY 165,052 after accounting for a CNY 6,720 increase in holding cost.

Beyond the numerical comparison in Table 4, the Business Analyst Agent (a_{BA}) converts the large-scale solver output for Scenario 2 into a structured business report. Specifically, it identifies the total cost as CNY 1,355,388 and attributes 68.26% to repositioning (CNY 925,238), 29.95% to leasing (CNY 406,000), and 1.78% to holding (CNY 24,150). It further highlights the dominant supply and receiving hubs, identifies shortage-prone inland nodes, and recommends prioritizing the Yingkou–Dalian repositioning corridor, securing long-term leasing agreements in inland shortage locations, and clearing stagnant inventories in peripheral nodes. This example illustrates how the Business Analyst Agent (a_{BA}) transforms a mathematically optimal solution into an actionable managerial interpretation.

5.4. Formulation Stability Analysis

This section examines whether the framework produces structurally consistent MIP formulations when applied repeatedly to the same problem instance. The standard benchmark datasets (ComplexOR and NLP4LP) used in Section 4 are

Table 4: Comparison of cost structures under the two scenarios (CNY)

Cost component	**Scenario 1**	**Scenario 2**	**Change**
Repositioning cost	**763,966**	**925,238**	−17.43%
Seaport feeder repositioning	58,296	60,328	−3.37%
Sea–land repositioning	594,952	864,910	−31.21%
Hinterland lateral repositioning	110,718	—	—
Holding cost	30,870	24,150	+27.83%
Leasing cost	395,500	406,000	−2.59%
Total cost	**1,190,336**	**1,355,388**	−12.18%

Note. Scenario 1 permits hinterland lateral repositioning; Scenario 2 disables inter-dry-port exchange.

designed for correctness evaluation: they involve relatively few variables and constraints, and the resulting formulation space is small. Under such conditions, the Structural-Agreement Rate (SAR), defined as pairwise agreement of the formulation structure $(n_{\text{vars}}, n_{\text{constrs}}, n_{\text{bin}}, n_{\text{int}}, \text{nnz})$, has limited discriminative power. Here, n_{vars} denotes the total number of decision variables, n_{constrs} the number of constraints, n_{bin} the number of binary variables, n_{int} the number of integer variables, and nnz the number of non-zero coefficients in the constraint matrix. Let $F_i = (n^i_{\text{vars}}, n^i_{\text{constrs}}, n^i_{\text{bin}}, n^i_{\text{int}}, \text{nnz}^i)$ denote the formulation structure observed in the i-th run that produced a valid formulation. The SAR is then computed as

$$\text{SAR} = \frac{\displaystyle\sum_{1 \le i < j \le R'} \mathbf{1}(F_i = F_j)}{\binom{R'}{2}}, \tag{11}$$

where R' is the number of runs that yielded a formulation structure (excluding only runs that failed before model construction) and $\mathbf{1}(\cdot)$ is the indicator function that equals 1 when the two formulation structures match exactly. When the formulation space is narrow, most structurally feasible formulations naturally coincide, yielding high SAR values that reflect problem simplicity rather than generation stability. Therefore, the empirical ECR instance used in this section is substantially more complex: 13 nodes (3 seaports and 10 dry ports) over 5 planning periods, yielding a formulation with approximately 975 variables and over 700 constraints in its fully

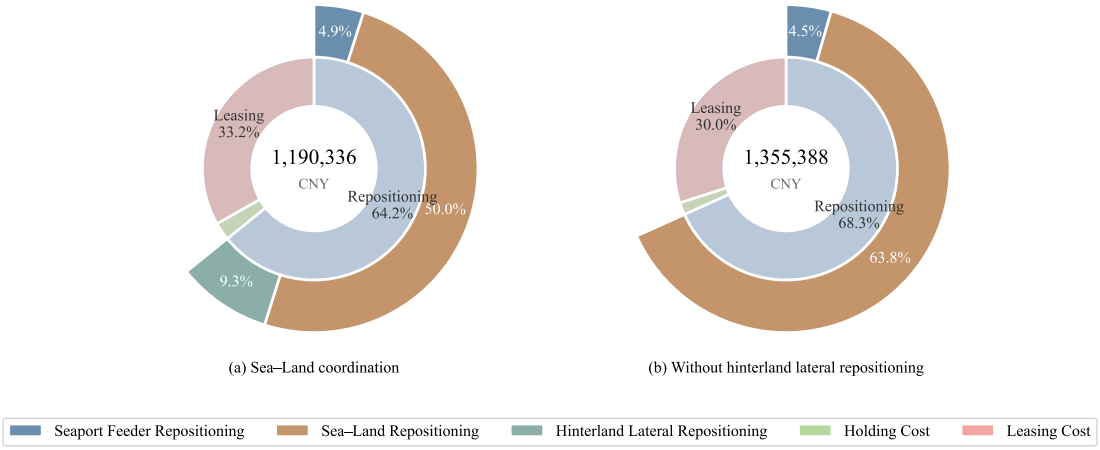


Figure 7: Cost comparison

specified form. This instance admits multiple structurally distinct yet feasible representations, where the formulation space is large enough to expose genuine variability in the generation process.

Table 5 summarizes the formulation stability results across two independent trials (R=25). The ground-truth optimal cost for Scenario 2 is CNY 1,355,388. Three patterns emerge from the results:

1. **Low SAR yet high AR.** The SAR is only 13.7%, with 8 distinct feasible formulation structures observed across 25 runs. Despite this structural diversity, all feasible runs converge to the same ground-truth optimum. The most frequent structure $(370, 130, 370, 718)$ accounts for 7 of 25 runs, while the remaining runs adopt alternative but mathematically equivalent encodings (e.g., variable upper bounds instead of explicit capacity constraints, or time-window pre-filtering of repositioning arcs). Here, "equivalent" means that the encodings represent the same logical constraints via different formulations (e.g., capacity as an explicit inequality versus a variable upper bound), and therefore induce the same optimum; it does not claim identical solver behavior. The structurally distinct encodings do differ in solve time and simplex iteration count, but these differences are bounded and negligible at this scale.

2. **Compact formulations dominate.** Among the 8 feasible structures, 6 use compact formulations with $n_{\text{vars}} \leq 458$, accounting for 15 of 18 feasible runs. Only 2 structures adopt the expanded formulation ($n_{\text{vars}} = 975$, creating vari-

ables for all node pairs), yet both achieve the correct objective by setting non-adjacent arc flows to zero.

3. **Error modes are coarse-grained.** In the observed runs, outcomes fall into two broad categories: feasible runs that all achieve the exact optimum, and code-execution failures (e.g., missing Gurobi license, syntax error) that prevent model construction entirely. Across 25 runs, the accuracy rate is 72.0% (18/25).

Table 5: Empirical ECR instance formulation stability.

ID	n_{vars}	n_{constrs}	n_{int}	nnz	Obj. (CNY)	Count	Status
—	—	—	—	—	—	5	Error
F1	361	0	361	0	—	1	Error
F2	370	0	370	0	—	1	Error
F3	361	65	361	644	1,355,388	1	Optimal
F4	361	130	361	709	1,355,388	2	Optimal
F5	370	65	370	653	1,355,388	3	Optimal
F6	370	74	370	662	1,355,388	1	Optimal
F7	370	130	370	718	1,355,388	7	Optimal
F8	458	221	458	884	1,355,388	1	Optimal
F9	975	130	975	1798	1,355,388	2	Optimal
F10	975	984	975	2652	1,355,388	1	Optimal
AR: $18/25 = 72.0\%$; **SAR:** $26/190 = 13.7\%$							

Note. $n_{\text{bin}} = n_{\text{int}}$ for all formulations, as every integer variable in the ECR model is binary; the column is therefore omitted to avoid redundancy.

The results reveal that the primary source of instability is *constraint-level encoding variability*: across independent runs, the LLM encodes the same logical constraints in structurally different ways. For example, the storage capacity constraint ("each node has a maximum storage capacity") is stated explicitly in the problem description. In F7 $(370, 130, 370, 718)$, the most frequent formulation with 7 runs, it appears as a standalone inequality ($I_{i,t} \leq U_i$, where $I_{i,t}$ is the inventory at node i in period t and U_i is the storage capacity); in F5 $(370, 65, 370, 653)$ (3 runs), the same bound is folded into the variable upper bound ($I_{i,t} \in [0, U_i]$) and therefore disappears from the constraint count. Both encodings are mathematically equivalent and yield the same optimum, yet they produce distinct formulation structures. This encoding variability, rather than constraint omission, accounts for most of the observed structural diversity.

6. Conclusions

In order to address the limitations associated with MIP, which predominantly relies on domain experts and requires problem-specific customization, this study introduces a decision-support framework titled SOP-MAS. This framework integrates large language models with multi-agent theory. The proposed framework facilitates end-to-end optimization by translating natural language descriptions of business optimization problems into actionable business solutions, thereby substantially reducing the application threshold of MIP. To mitigate hallucination issues and the black-box nature of LLM-based decision-making, this study introduces carefully designed prompts, an orchestrated workflow based on standard operating procedures, and an exception-backtracking mechanism to enhance interpretability.

The effectiveness of the approach was validated through comparisons with multiple baseline methods on the ComplexOR and NLP4LP benchmark datasets, achieving an accuracy of 86.59%. SOP-MAS was subsequently employed to address the empty container repositioning challenge in the Liaoning coastal port cluster–Northeast hinterland context, leading to a 12.18% reduction in total operational costs for the shipping company. This application notably curtailed modeling and solution times, expedited prototype development, and furnished scalable decision support amid evolving business rules.

SOP-MAS is deliberately designed as an expert-in-the-loop decision support system, where human expertise constitutes a core architectural layer rather than a residual limitation. Domain experts retain ultimate responsibility for model validity, constraint fidelity, and operational feasibility. Several promising directions merit further investigation. First, extending the SOP-MAS framework to more complex optimization paradigms, such as stochastic programming, robust optimization, and multi-objective optimization, would broaden its applicability to real-world ECR scenarios involving demand uncertainty, disruption risks, and conflicting stakeholder objectives. Second, deploying the framework in interactive, human-in-the-loop settings would enable iterative refinement and support a broader set of stakeholders, including seaports, dry ports, shippers, and third-party leasing companies, within the container supply chain. Third, while the evaluation metrics rigorously assess the optimization core of SOP-MAS, the Business Analyst Agent (a_{BA}) introduces a qualitatively different output type, namely business interpretation and decision recommendations, that extends beyond solution correctness. Developing dedicated evaluation dimensions for this agent, such as interpretation accuracy, recommendation completeness, and decision actionability, and validating them through structured expert studies with logistics practitioners, would enrich the multi-agent evaluation framework and provide rigorous empirical evidence of the Business Analyst Agent

(a_{BA})'s contribution to decision support.

Data Availability

The code and data that support the findings of this study are publicly available in the SOP-MAS repository at `https://github.com/icsco-mako/sop-mas-ecrp`.

Appendix A. Agent Prompt Template

We present the prompt template of the core agent (the Modeling Expert Agent, a_{ME}) as a representative example. Complete templates for all agents are available at `https://github.com/icsco-mako/sop-mas-ecrp`.

```
# ROLE: Modeling Expert

## INPUT
- Problem description: {problem_description}
- Expert comments: {message_pool}
- Naming convention: {parameter_naming_convention_document}

## TASK
Build a complete mathematical optimization model through:

1. Problem Analysis
 Identify problem type (LP / IP / MIP) from inputs.

2. Model Construction
 (a) Decision Variables
 Define all variables with CamelCase naming.
 Annotate types (Continuous / Integer / Binary).
 (b) Objective Function
 Formulate with explicit unit labels.
 Verify dimensional consistency with constraints.
 (c) Constraints
 Categorize as Resource / Logical / Business.
 List non-negativity constraints separately for
 each variable type; do not combine them.
 Use non-strict inequalities only (<=, >=, =).

3. Validation Rules
 * No new parameters may be introduced.
 * Auxiliary variables are allowed for model clarity.
 * Express all math in LaTeX notation ($x$, $$eq$$).
 * No code generation required.

## OUTPUT FORMAT (Markdown)
# parameters
(list all parameters with symbols and definitions)

# variables
(list all variables with types and dimensions)

# objective
(the objective function in LaTeX)

# constraints
(all constraints in LaTeX, grouped by category)

# ambiguities
(any unclear points from the problem description)
```

Listing 1: Representative prompt template for the Modeling Expert Agent (a_{ME})

Appendix B. Empirical Case Data

The case-study network comprises three seaports (Dalian, Yingkou, Dandong) and ten inland cities served by dry ports. Inter-node distances and sea-repositioning transit times are standard road and sea values, whose full matrices are provided in the public SOP-MAS repository; the unit repositioning cost (0.8 CNY/TEU/km) and all other scalar parameters are reported in Section 5.2. Table B.6 reports the per-node empty-container demand and supply, each modeled as a normal distribution characterized by its mean and variance.

Table B.6: Empty container demand and supply at each node (TEU)

Port/EC volume (TEU)	EC demand	EC supply
Shenyang	$(5000, 100^2)$	$(4150, 100^2)$
Anshan	$(1250, 50^2)$	$(1350, 50^2)$
Harbin	$(800, 40^2)$	$(800, 40^2)$
Changchun	$(500, 20^2)$	$(500, 20^2)$
Tonghua	$(500, 20^2)$	$(500, 20^2)$
Jilin	$(500, 20^2)$	$(500, 20^2)$
Tongliao	$(500, 20^2)$	$(500, 20^2)$
Chifeng	$(100, 20^2)$	$(100, 20^2)$
Daqing	$(300, 20^2)$	$(300, 20^2)$
Qiqihar	$(100, 20^2)$	$(100, 20^2)$

Appendix C. ECR Mathematical Model

As one of our core contributions is the automatic synthesis of publication-ready mathematical formulations, we present the complete ECR model generated by the SOP-MAC framework below. The main text summarizes the problem setting, key decision logic, and operational assumptions in Section 5.2, while this appendix provides the full formulation for readers seeking model-level detail.

```
### **Parameters**

- $ \mathcal{P} $: Set of seaports
- $ \mathcal{L} $: Set of inland ports
- $ \mathcal{N} = \mathcal{P} \cup \mathcal{L} $: Set of nodes
- $ \mathcal{M} $: Set of transportation modes (e.g., ship, truck, train)
- $ T $: Set of time periods ($ T = \{1, 2, \ldots, |T|\} $)
- $ \lambda_{ij} \in \{0,1\} $: $ = 1 $ if inland port $ j $ belongs to hinterland of seaport $
    i $
- $ \alpha_{j_1 j_2} \in \{0,1\} $: $ = \max_{h \in \mathcal{P}} (\lambda_{h j_1} \lambda_{h
    j_2}) $, inland port collaboration indicator
- $ Q_i^t \geq 0 $: Empty container supply at node $ i \in \mathcal{N} $ in period $ t $
- $ D_i^t \geq 0 $: Empty container demand at node $ i \in \mathcal{N} $ in period $ t $
- $ U_i^{\max} > 0 $: Maximum storage capacity at node $ i \in \mathcal{N} $
- $ c_i^{\text{hold}} > 0 $: Unit inventory holding cost at node $ i \in \mathcal{N} $
- $ c_{ijm}^{\text{repos}} > 0 $: Unit repositioning cost for moving an empty container from $
    i $ to $ j $ via mode $ m $
- $ F_{ijm}^{\max} \geq 0 $: Maximum transportation capacity on arc $ (i,j) $ using mode $ m $
- $ c_i^{\text{rent}} > 0 $: Unit container rental cost at node $ i \in \mathcal{N} $
- $ \tau_{ij} \geq 1 $: Repositioning lead time (in periods) for shipments from seaport $ i $
    to seaport $ j $ (defined only for $ i,j \in \mathcal{P} $)
- $ w_i^0 \geq 0 $: Initial inventory at node $ i $ at the beginning of period 1
- $ M = \sum_{t \in T} \sum_{i \in \mathcal{N}} D_i^t $: A sufficiently large constant

### **Decision Variables**

- $ w_i^t \geq 0 $: Inventory of empty containers at node $ i \in \mathcal{N} $ at the end of
    period $ t $ (continuous)
- $ x_{ijm}^t \geq 0 $: Quantity of empty containers repositioned from node $ i $ to node $ j $
    via mode $ m $ in period $ t $ (continuous)
- $ r_i^t \geq 0 $: Quantity of containers rented at node $ i \in \mathcal{N} $ in period $ t $
    (continuous)

### **Objective Function**
Minimize total operational cost (repositioning + holding + rental):
$$
\min \sum_{t \in T} \left(
\sum_{i \in \mathcal{N}} \sum_{j \in \mathcal{N}} \sum_{m \in \mathcal{M}}
    c_{ijm}^{\text{repos}} x_{ijm}^t
+ \sum_{i \in \mathcal{N}} c_i^{\text{hold}} w_i^t
+ \sum_{i \in \mathcal{N}} c_i^{\text{rent}} r_i^t
\right)
$$

### **Constraints**
```

```
#### **1. Flow Balance Constraints**
**For seaport nodes $ i \in \mathcal{P} $** (incorporating hinterland affiliation and lead
    time):
$$
w_i^{t-1} + Q_i^t + r_i^t + \sum_{\substack{j \in \mathcal{P} \\ m \in \mathcal{M} \\ t >
    \tau_{ji}}} x_{jim}^{t - \tau_{ji}} + \sum_{\substack{j \in \mathcal{L} \\ m \in
    \mathcal{M} \\ \lambda_{ij} = 1}} x_{jim}^{t} = D_i^t + \sum_{\substack{j \in \mathcal{P}
    \\ m \in \mathcal{M}}} x_{ijm}^{t} + \sum_{\substack{j \in \mathcal{L} \\ m \in \mathcal{M}
    \\ \lambda_{ij} = 1}} x_{ijm}^{t} + w_i^{t} \quad \forall t \in T
$$

**For inland port nodes $ j \in \mathcal{L} $** (strict hinterland restriction; all
    repositioning is instantaneous):
$$
w_j^{t-1} + Q_j^t + r_j^t + \sum_{\substack{i \in \mathcal{P} \\ m \in \mathcal{M} \\
    \lambda_{ij} = 1}} x_{ijm}^{t} + \sum_{\substack{i \in \mathcal{L} \\ m \in \mathcal{M} \\
    \alpha_{ij} = 1}} x_{ijm}^{t} = D_j^t + \sum_{\substack{i \in \mathcal{P} \\ m \in
    \mathcal{M} \\ \lambda_{ij} = 1}} x_{jim}^{t} + \sum_{\substack{i \in \mathcal{L} \\ m \in
    \mathcal{M} \\ \alpha_{ij} = 1}} x_{jim}^{t} + w_j^{t} \quad \forall t \in T
$$

#### **2. Hinterland Logic Constraints**
**Seaport and inland port repositioning** (forbidden outside designated hinterlands):
$$
x_{ijm}^t = 0 \quad \forall i \in \mathcal{P}, j \in \mathcal{L}, m \in \mathcal{M}, t \in T,
    \text{ if } \lambda_{ij} = 0
$$

$$
x_{jim}^t = 0 \quad \forall i \in \mathcal{P}, j \in \mathcal{L}, m \in \mathcal{M}, t \in T,
    \text{ if } \lambda_{ij} = 0
$$

**Inland port and inland port repositioning** (permitted only if ports share a common seaport
    hinterland):
$$
x_{j_1 j_2 m}^t \leq M \cdot \alpha_{j_1 j_2} \quad \forall j_1, j_2 \in \mathcal{L}, m \in
    \mathcal{M}, t \in T
$$

#### **3. Physical Limit Constraints**
**Transportation capacity constraints** (for all arcs):
$$
x_{ijm}^t \leq F_{ijm}^{\max} \quad \forall i,j \in \mathcal{N}, m \in \mathcal{M}, t \in T
$$

**Storage capacity constraints** (for all nodes):
$$
w_i^t \leq U_i^{\max} \quad \forall i \in \mathcal{N}, t \in T
$$

**Non-negativity constraints** (for all variables):
$$
w_i^t \geq 0, \quad x_{ijm}^t \geq 0, \quad r_i^t \geq 0 \quad \forall i,j \in \mathcal{N}, m
    \in \mathcal{M}, t \in T
$$
```

Appendix D. Case Analysis of Formulation Variability

To ground the aggregate findings of the formulation stability experiment (Section 6.3), we analyze a representative run (F10) from the 25 independent trials, which is correct despite a structurally unusual formulation.

F10 yields the structure (975, 984, 975, 2652), the largest among all 25 runs, yet achieves the ground-truth optimum of CNY 1,355,388. The key difference is in how repositioning variables are defined. Instead of creating variables only for adjacent arcs (the approach taken by F7, the most common formulation with 7 runs), the generated code enumerates *all* node pairs and enforces zero flow on non-adjacent arcs through big-M constraints:

```
# Variables: reposition for ALL node pairs (13x13x5 = 845)
reposition = m.addVars(N, N, T, vtype=GRB.INTEGER, lb=0)
inventory = m.addVars(N, T, vtype=GRB.INTEGER, lb=0) # 65
leasing = m.addVars(N, T, vtype=GRB.INTEGER, lb=0) # 65

# Big-M: force zero on non-adjacent arcs
bigM = max(sum(supply[i]) + sum(demand[i]) for i in range(N)) * 10
for i in range(N):
 for j in range(N):
 for t in range(T):
 m.addConstr(reposition[i,j,t] <= adj_matrix[i][j] * bigM)
 if t + tau_matrix[i][j] >= T: # arrival beyond horizon
 m.addConstr(reposition[i,j,t] == 0)
```

This encoding produces $845 + 65 + 65 = 975$ variables and $65 + 65 + 845 + 9 = 984$ constraints (balance, capacity, adjacency, and time-window), with 2,652 non-zero coefficients. Despite the structural inflation, the solver correctly identifies that all non-adjacent arc flows must be zero and converges to the same optimum as compact formulations. This case illustrates that LLM-generated formulations can be *structurally different yet mathematically equivalent*: the choice of encoding strategy (full enumeration with big-M vs. sparse variable filtering) does not affect solution correctness.

References

[1] J. Hu, X. Ai, J. Wang, The impact of supply chain digitalization on the technological innovation of new energy vehicle enterprises: the moderated mediation effect, Business Process Management Journal (2026). URL: https://api.semanticscholar.org/CorpusID:287297206.

[2] X. Xiang, X. Xu, C. Liu, S. Jia, Liner fleet deployment and empty container repositioning under demand uncertainty: A robust optimization approach, Transportation Research Part B: Methodological 190 (2024) 103088. doi:10.1016/j.trb.2024.103088.

[3] X. Xiang, C. Liu, S. Jia, Tactical vessel deployment and empty container repositioning considering container turnover times, Computers & Industrial Engineering 190 (2024) 110032. doi:10.1016/j.cie.2024.110032.

[4] X. Xu, X. Huang, L. Wang, Empty container repositioning problem using a reinforcement learning framework with multi-weight adaptive reward function, Maritime Policy & Management (2024) 1–22. doi:10.1080/03088839.2024.2326635.

[5] K. A. Kuźmicz, E. Pesch, Container depot location problem in the frame of the Polish part of the New Silk Road, MATEC Web of Conferences 312 (2020) 02004. doi:10.1051/matecconf/202031202004.

[6] W. Shan, Z. Peng, J. Liu, B. Yao, B. Yu, An exact algorithm for inland container transportation network design, Transportation Research Part B: Methodological 135 (2020) 41–82. doi:10.1016/j.trb.2020.02.011.

[7] H. Zhang, L. Lu, X. Wang, Tactical and Operational Cooperative Empty Container Repositioning Optimization Model Based on Business Flow and Initial Solutions Generation Rules, Symmetry 11 (2019) 300. doi:10.3390/sym11030300.

[8] J. Liang, Z. Ma, S. Wang, H. Liu, Z. Tan, Dynamic container slot allocation with empty container repositioning under stochastic demand, Transportation Research Part E: Logistics and Transportation Review 187 (2024) 103603. doi:10.1016/j.tre.2024.103603.

[9] M. Yu, X. Tan, J. Chen, Pricing competition in maritime transportation with blockchain technology and empty container repositioning, Advanced Engineering Informatics 59 (2024) 102318. doi:10.1016/j.aei.2023.102318.

[10] S. Fazi, S. K. Choudhary, J.-X. Dong, The multi-trip container drayage problem with synchronization for efficient empty containers re-usage, European Journal of Operational Research 310 (2023) 343–359. doi:10.1016/j.ejor.2023.02.041.

[11] J. Cai, Y. Huang, C. Diao, Z. Jin, Joint optimization of empty container repositioning and inventory control applying dynamic programming and simulated annealing, Applied Soft Computing 167 (2024) 112452. doi:10.1016/j.asoc.2024.112452.

[12] S. R. Addula, A. Ali, A novel permissioned blockchain approach for scalable and privacy-preserving IoT authentication, Journal of Cyber Security and Risk Auditing 2025 (2025) 222–237. doi:10.63180/jcsra.thestap.2025.4.3.

[13] T. Luo, D. Chang, Empty container repositioning strategy in intermodal transport with demand switching, Advanced Engineering Informatics 40 (2019) 1–13. doi:10.1016/j.aei.2019.02.008.

[14] Y. Xie, X. Liang, L. Ma, H. Yan, Empty container management and coordination in intermodal transport, European Journal of Operational Research 257 (2017) 223–232. doi:10.1016/j.ejor.2016.07.053.

[15] B. Li, K. Mellou, B. Zhang, J. Pathuri, I. Menache, Large Language Models for Supply Chain Optimization, 2023. doi:10.48550/arXiv.2307.03875. arXiv:2307.03875.

[16] B. Zhang, H. Sang, L. Meng, X. Jiang, C. Lu, Knowledge- and data-driven hybrid method for lot streaming scheduling in hybrid flowshop with dynamic order arrivals, Computers & Operations Research 184 (2025) 107244. doi:10.1016/j.cor.2025.107244.

[17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22, Curran Associates Inc., Red Hook, NY, USA, 2022, pp. 24824–24837.

[18] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan, Tree of Thoughts: Deliberate Problem Solving with Large Language Models, Advances in Neural Information Processing Systems 36 (2023) 11809–11822.

[19] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, T. Hoefler, Graph of thoughts: Solving elaborate problems with large language models, in: Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, volume 38 of *AAAI'24/IAAI'24/EAAI'24*, AAAI Press, 2024, pp. 17682–17690. doi:10.1609/aaai.v38i16.29720.

[20] P. Li, Q. Yang, S. Xu, X. Li, Z. Li, C. Wang, Y. Liu, T. Guo, J. Tang, Y. Wen, Adaptive-TOD: An LLM-driven and adaptive agent for diverse interaction modes, Neurocomputing 652 (2025) 130991. doi:10.1016/j.neucom.2025.130991.

[21] X. Jiang, Y. Wu, C. Zhang, Y. Zhang, DROC: ELEVATING LARGE LANGUAGE MODELS FOR COMPLEX VEHICLE ROUTING VIA DECOMPOSED RE- TRIEVAL OF CONSTRAINTS (2025).

[22] C. Liu, K. H. Hettige, Q. Xu, C. Long, S. Xiang, G. Cong, Z. Li, R. Zhao, ST-LLM+: Graph Enhanced Spatio-Temporal Large Language Models for Traffic Prediction, IEEE Transactions on Knowledge and Data Engineering 37 (2025) 4846–4859. doi:10.1109/TKDE.2025.3570705.

[23] Y. Ishibashi, Y. Nishimura, Self-Organized Agents: A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization, 2024. doi:10.48550/arXiv.2404.02183. arXiv:2404.02183.

[24] W. Yuan, J. Chen, S. Chen, D. Feng, Z. Hu, P. Li, W. Zhao, Transformer in reinforcement learning for decision-making: A survey, Frontiers of Information Technology & Electronic Engineering 25 (2024) 763–790. doi:10.1631/FITEE.2300548.

[25] X. Li, J. Gu, F. He, Y. Xiao, Y. Jia, P. Lan, Multitype game optimisation: A two-stage fine-tuning framework for multi-game optimisation with large language models, CAAI Transactions on Intelligence Technology 11 (2026) 739–753. URL: https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/cit2.70108. doi:https://doi.org/10.1049/cit2.70108. arXiv:https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/cit2.70108.

[26] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A Survey on Hallucination in Large Language Models:

Principles, Taxonomy, Challenges, and Open Questions, ACM Transactions on Information Systems 43 (2025) 1–55. doi:10.1145/3703155.

[27] P. Laban, H. Hayashi, Y. Zhou, J. Neville, LLMs Get Lost In Multi-Turn Conversation, 2025. doi:10.48550/arXiv.2505.06120. arXiv:2505.06120.

[28] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations, in: First Conference on Language Modeling, 2024.

[29] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework, in: International Conference on Learning Representations, 2023.

[30] D. Zhang, Z. Xiao, Y. Wang, M. Song, G. Chen, Neural TSP Solver with Progressive Distillation, in: B. Williams, Y. Chen, J. Neville (Eds.), Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023, AAAI Press, 2023, pp. 12147–12154. doi:10.1609/AAAI.V37I10.26432.

[31] R. Ramamonjison, T. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabdaveh, A. Banitalebi-Dehkordi, Z. Zhou, Y. Zhang, NL4Opt Competition: Formulating Optimization Problems Based on Their Natural Language Descriptions, in: Proceedings of the NeurIPS 2022 Competitions Track, PMLR, 2023, pp. 189–203.

[32] A. Ahmaditeshnizi, W. Gao, M. Udell, OptiMUS: Scalable Optimization Modeling with (MI)LP Solvers and Large Language Models, in: Proceedings of the 41st International Conference on Machine Learning, PMLR, 2024, pp. 577–596.

[33] Z. Xiao, D. Zhang, Y. Wu, L. Xu, Y. J. Wang, X. Han, X. Fu, T. Zhong, J. Zeng, M. Song, G. Chen, Chain-of-Experts: When LLMs Meet Complex Operations Research Problems, in: The Twelfth International Conference on Learning Representations, 2023.